Relatório: UberPB

1. **O backlog com os itens de trabalho e as tasks;**

Para apresentação do backlog e as task será utilizado prints do Azure DevOps

- **Sprint 1 (26/08 – 08/09)**

Cadastro e autenticação	Lucas Nascime...
CRUD de Passageiros	Lucas Nascime...
Persistência dos dados	Lucas Nascime...
Implementação de login do sistema	Mateus Rufino ...
Implementação das categorias de veículos	Manoel manoel
CRUD de Motorista	Lucas Nascime...
CRUD de Gerente	Lucas Nascime...
Solicitação de corrida	Lucas Nascime...
Implementar calculo de preço	Erick Nathan
Implementar escolha de categoria para o usuário	Mateus Rufino ...
Implementação da classe corrida	Mateus Rufino ...
Implementação de estimativas da corrida	Erick Nathan
Implementar notificação aos motoristas selecionaveis par...	Mateus Rufino ...
Testes de Unidade	Lucas Nascime...
Teste das funcionalidades de cadastro e autenticação	Erick Nathan
Teste das funcionalidades de solicitação de corrida	Erick Nathan
Documentação	Lucas Nascime...
Documentação das funcionalidades de Cadastro e autent...	Manoel manoel
Documentação das funcionalidades de solicitação de cor...	Manoel manoel

Figura 1

Durante a Sprint 1, a equipe concentrou-se no desenvolvimento inicial das principais funcionalidades do sistema, garantindo a base para as próximas entregas. As atividades realizadas foram:

- **Cadastro e autenticação:** CRUD de Passageiros, Persistência dos dados, Implementação de login do sistema, Implementação das categorias de veículos, CRUD de Motorista e CRUD de Gerente.
- **Solicitação de corrida:** Cálculo de preço, Escolha de categoria para o usuário, Classe corrida, Estimativas da corrida e Notificação aos motoristas selecionados.
- **Testes de unidade:** Testes das funcionalidades de cadastro/autenticação e solicitação de corrida.
- **Documentação:** Registro das funcionalidades de Cadastro e Autenticação, além da Solicitação de Corrida.

Todos os itens planejados foram concluídos com sucesso, permitindo validar as primeiras interações entre usuários e o sistema.

- **Papéis**
  - **Erick:** Testador
  - **Lucas:** Gerente, Desenvolvedor
  - **Manoel:** Desenvolvedor, Engenheiro de software
  - **Mateus:** Desenvolvedor

Como foi implementado (Cadastro e Autenticação): Foram criadas as classes de entidade (`Passageiro`, `Motorista`, `Gerente`), os Repositórios (`PassageiroRepository`, `MotoristaRepository`) e os Serviços (`PassageiroService`, `MotoristaService`) para lidar com as regras de negócio e a persistência em arquivos JSON. A enum `CategoriaVeiculo` foi criada e o `LoginView` foi implementado para autenticação.

Como foi implementado (Solitação de corrida): A `Corrida.java` foi criada. A lógica de `cálculo de preço` foi adicionada ao `CorridaService`. A `SolicitarCorridaView` foi criada para permitir ao usuário inserir origem/destino e escolher a categoria. A `notificação` foi implementada no método `encontrarProximoMotoristaDisponivel` do `CorridaService`.

- **Sprint 2 (09/09 – 22/09)**

Documentação	Mateus Rufino ...
Documentação das funcionalidades de Aceite da corrida	Manoel manoel
Documentação das funcionalidades do Acompanhament...	Manoel manoel
Acompanhamento da Corrida	Mateus Rufino ...
Confirmação de corrida do motorista	Mateus Rufino ...
Visualização da localização do passageiro	Lucas Nascime...
Estimativa da chegada do motorista	Lucas Nascime...
Implementar visualização da melhor rota até o destino	Lucas Nascime...
Aceite da Corrida	Mateus Rufino ...
Atribuir corrida ao motorista mais próximo	Mateus Rufino ...
Refatoração	Mateus Rufino ...
Aprimorar documentação de veículo	Manoel manoel
Interface do Usuário	Mateus Rufino ...
Implementação da Interface CLI	Mateus Rufino ...
Testes de Unidade	Mateus Rufino ...
Testes das funcionalidades de Aceite da Corrida	Erick Nathan
Testes das funcionalidades de Acompanhamento da Corr...	Erick Nathan

Figura 2

Na Sprint 2, o foco foi ampliar as funcionalidades já desenvolvidas, adicionando camadas de acompanhamento e aceitação da corrida, além de melhorias na interface e na documentação. As atividades realizadas foram:

- **Documentação:** Funcionalidades de Aceite da Corrida e Acompanhamento da Corrida.
- **Acompanhamento da Corrida:** Estimativa da chegada do motorista e visualização da melhor rota até o destino.
- **Aceite da Corrida:** Confirmação da corrida pelo motorista e atribuição da corrida ao motorista mais próximo.
- **Refatoração:** Melhoria da documentação de veículos.
- **Interface do Usuário:** Implementação da Interface CLI.

- **Testes de unidade:** Testes das funcionalidades de Aceite da Corrida e Acompanhamento da Corrida.

Grande parte das tarefas foi concluída, enquanto as atividades com os requisitos com funcionalidade de localização do motorista ainda faltam terminar a implementação.

- **Papéis**
  - **Erick:** Testador
  - **Lucas:** Desenvolvedor
  - **Manoel:** Desenvolvedor, Engenheiro de software
  - **Mateus:** Gerente, Desenvolvedor

Como foi implementado (Aceite corrida): A lógica de atribuição foi implementada no `CorridaService`, que filtra motoristas por status e categoria, e os ordena por distância/avaliação. A confirmação foi implementada nos métodos `aceitarCorrida` e `negarCorrida`, acessíveis ao motorista pela `CorridaView`.

Como foi implementado (Acompanhamento da corrida): A visualização da rota foi implementada através do `SimuladorViagem.java`, que atualiza um `dados_viagem.js` e é exibido no `Mapa.html`. A `AcompanharCorridasView` foi criada para o passageiro ver o status.

- **Sprint 3 (25/09 – 08/10)**

Refatoração	Manoel manoel
Requisitos com funcionalidade de localização do motorista	Lucas Nascime...
Pagamentos	Manoel manoel
Emitir recibo eletrônico	Lucas Nascime...
Calcular o valor da corrida	Mateus Rufino ...
Pagamento via cartão, paypal ou dinheiro	Mateus Rufino ...
Testes de unidade	Manoel manoel
Teste de unidade da funcionalidade emitir recibo eletrôni...	Erick Nathan
Teste de unidade da funcionalidade calcular valor da corr...	Erick Nathan
Teste de unidade da funcionalidade pagamento	Erick Nathan
Documentação	Manoel manoel
Documentação da funcionalidade emitir recibo eletrônico	Manoel manoel
Documentação de calcular valor da corrida	Manoel manoel

Figura 3

Durante a Sprint 3, o foco da equipe esteve na implementação das funcionalidades relacionadas a **pagamentos**, **refatoração de código**, **testes de unidade** e **documentação**. Todas as atividades planejadas foram concluídas dentro do prazo estabelecido, garantindo a continuidade do desenvolvimento com qualidade e estabilidade.

As principais entregas desta sprint foram:

- **Refatoração**: Ajustes e melhorias no código, incluindo a adição de requisitos com a funcionalidade de localização do motorista.
- **Pagamentos**: Implementação das funcionalidades de emissão de recibo eletrônico, cálculo do valor da corrida e integração com múltiplas formas de pagamento (cartão, PayPal e dinheiro).
- **Testes de unidade**: Desenvolvimento e execução de testes para validar as funcionalidades de emissão de recibo eletrônico, cálculo de valor da corrida e pagamento.
- **Documentação**: Registro técnico das funcionalidades implementadas, abrangendo pagamento, cálculo de valor da corrida e emissão de recibo eletrônico.

A sprint foi finalizada com 100% das atividades concluídas, consolidando uma base sólida para a próxima etapa do projeto, com foco em garantir qualidade e manutenção das novas funcionalidades.

- **Papéis**
  - **Erick:** Testador
  - **Lucas:** Desenvolvedor
  - **Manoel:** Gerente, Engenheiro de software
  - **Mateus:** Desenvolvedor

Como foi implementado (Refatoração): A lógica de simulação de localização do motorista foi concluída no `MotoristaService` e `CorridaView`, permitindo ao motorista "mudar" de local e receber corridas com base nessa nova posição.

Como foi implementado: A `PagamentoView.java` foi criada para processar as formas de pagamento . Além das formas citadas na *task*, Pix também foi implementado. O Cálculo do valor foi refinado no `CorridaService`. O recibo eletrônico é apresentado ao usuário no `HistoricoCorridasView`.

- **Sprint 4 (9/10 – 22/10)**

Sistema de Avaliações	Erick Nathan
Definir avaliação entre passageiros e motoristas mutuam...	Mateus Rufino ...
Relacionamento entre usuários e feedback	Lucas Nascime...
Cálculo de indicadores	Lucas Nascime...
Interface para consulta e envio de feedback	Mateus Rufino ...
Definir decisão de prioridade	Lucas Nascime...
Definir categorias de serviço	Mateus Rufino ...
Implementar cálculo de relevância das avaliações	Erick Nathan
Definir métricas de desempenho para medir a qualidade ...	Manoel manoel

Figura 4

Sistema de Histórico	Erick Nathan
Definir estrutura de registro de viagens.	Lucas Nascime...
Implementar lógica de mecanismo de filtragem.	Lucas Nascime...
Adicionar à interface a de consulta de histórico	Mateus Rufino ...
Estabelecer categorias relevantes de veículos ou serviços.	Mateus Rufino ...
Documentação	Erick Nathan
Documentação do sistema de avaliação	Manoel manoel
Documentação do sistema de histórico	Manoel manoel
Testes	Erick Nathan
Teste do sistema de históricos	Erick Nathan
Teste do sistema de avaliações	Erick Nathan

Figura 5

Durante a Sprint 4, a equipe concentrou seus esforços no desenvolvimento e documentação das novas funcionalidades de **avaliações** e **histórico de corridas**, ampliando a experiência dos usuários e aprimorando a interação entre passageiros e motoristas. Todas as atividades planejadas foram concluídas com sucesso, garantindo

a entrega completa dos requisitos definidos.
As principais entregas desta sprint foram:

- **Avaliação**: Foi desenvolvida a funcionalidade que permite que passageiros e motoristas realizem avaliações mútuas após o término de cada corrida. Além disso, as médias de avaliação passaram a ser consideradas no momento de priorização dos motoristas.
- **Histórico**: Implementação do histórico de corridas, que possibilita o registro completo das viagens realizadas pelos usuários. A funcionalidade permite filtrar corridas por categoria de veículo, facilitando a consulta e o acompanhamento de viagens anteriores.
- **Documentação**: Registro técnico das funcionalidades implementadas, abrangendo avaliação mútua, priorização dos motoristas por avaliação e o histórico das corridas.

A sprint foi finalizada com todas as atividades concluída**s**, garantindo estabilidade e ampliando as funcionalidades do sistema com foco em usabilidade e qualidade.

- **Papéis**
  - **Erick:** Gerente, Testador
  - **Lucas:** Desenvolvedor
  - **Manoel:** Engenheiro de software
  - **Mateus:** Desenvolvedor

Como foi implementado (Avaliação): A avaliação mútua foi implementada nas classes `AvaliacaoView` e `AvaliacaoPendentesView`. A priorização de motoristas foi implementada no `CorridaService`, que usa um comparador para ordenar motoristas com base na média de avaliação. A média é recalculada e salva pelos métodos `recalcularMediaAvaliacao` nos serviços `MotoristaService` e `PassageiroService`.

Como foi implementado (Histórico): A Interface de consulta e a lógica de filtragem por categoria foram implementadas na classe `HistoricoCorridasView`. A estrutura de registro é a lista `historicoCorridas` na entidade `Passageiro.java`.

- **Sprint 5 (05/02 – 23/02)**

Documentação	
Atualizar documentação com as novas entidades criadas	Johnatan Rodrigues
Testes	
Realizar testes para as funcionalidade da Solicitação de P...	antonio neri
Implementar testes para as funcionalidades de CRUD de ...	antonio neri
Solicitação de Pedidos	
Preparar CLI para receber pedidos de novos usuários	Lucas Nascimento
Permitir montagem do pedido	Lucas Nascimento
Implementar agendamento de pedido	Lucas Nascimento
Criar as entidades do Negócio (Cardápio)	Lucas Nascimento
CRUD de Entidades	
Criar novas entidades	Johnatan Rodrigues
Autenticação de novas entidades	Johnatan Rodrigues
Preparar CLI para login dos novos usuários	Johnatan Rodrigues

Figura 6

Durante a Sprint 5, a equipe concentrou seus esforços no desenvolvimento orgânico do módulo de delivery, focando nas funcionalidades de Solicitação de Pedidos e no CRUD das novas entidades (Entregadores e Restaurantes), além de manter a documentação atualizada e realizar as baterias de testes. Todas as atividades planejadas foram concluídas com sucesso, estruturando a base necessária para a expansão do sistema.

As principais entregas desta sprint foram:

- **CRUD de Entidades:** Implementação completa da criação, manipulação e autenticação das entidades Restaurante e Entregador, além de preparar a interface de linha de comando (CLI) para o login desses novos usuários.

- **Solicitação de Pedidos:** Desenvolvimento do fluxo de pedidos, o que inclui a criação da entidade Cardápio, a possibilidade de montagem e agendamento de pedidos, e a preparação da CLI para suportar a recepção de pedidos dos novos usuários.
- **Testes:** Realização e implementação de rotinas de testes garantindo o funcionamento do CRUD de entidades e da nova lógica de Solicitação de Pedidos.
- **Documentação:** Atualização contínua dos diagramas de documentação do sistema para refletir a nova arquitetura abordando Entregadores, Restaurantes e Cardápio.

A sprint foi finalizada com todas as atividades concluídas, garantindo estabilidade e expandindo os casos de uso para envolver uma experiência de delivery escalável e de qualidade.

- **Papéis:**
  - **Antônio Neri:** Testador
  - **Johnatan Rodrigues:** Desenvolvedor
  - **Lucas Nascimento**: Gerente, Desenvolvedor

A modelagem inicial consistiu na criação das entidades Entregador e Restaurante. O acesso e salvamento de dados foram garantidos através dos repositórios (`EntregadorRepository` e `RestauranteRepository`). O encapsulamento das lógicas de negócio foi programado nos serviços (`EntregadorService` e *RestauranteService*). A interface e fluxos das entidades no terminal, bem como o processo de login ou criação de conta, foram gerenciadas através das Views correspondentes (como `CadastroEntregadorView` e `LoginView`). Como foi implementado (Solicitação de Pedidos): A lógica para a escolha de pratos e agendamento foi agregada com a criação da entidade Cardápio. Para a consolidação do pedido via CLI, foram criados menus interativos na View que gerenciam as entradas do usuário e processam as regras de montagem e consolidação do pedido no sistema.

- **Sprint 6 (24/02 – 19/02)**

Documentação	Johnatan Rodrigues
Atualizar diagrama de classes e diagramas de caso de uso	Johnatan Rodrigues
Atualizar diagrama de atividade	Johnatan Rodrigues
Atualizar relatórios da release	Johnatan Rodrigues
Testes	Johnatan Rodrigues
Implementar testes unitários para a entidade e repositório de Avaliações.	Lucas Nascimento
Realizar testes de integração no fluxo de aceite/recusa de pedidos	Lucas Nascimento
Aprimorar testes feitos anteriormente	Lucas Nascimento
Sistema de Avaliações	Johnatan Rodrigues
Criar entidades e repositório/estrutura de dados para armazenar Avaliações (Rating/Reviews).	antonio neri
Implementar CLI para o Cliente avaliar o Restaurante e o Entregador após a entrega.	antonio neri
Implementar CLI para o Entregador e Restaurante avaliarem a experiência.	antonio neri
Acompanhamento e Rota de Entrega	Johnatan Rodrigues
Implementar lógica de visualização/simulação de Rota e Mapa para o Entregador.	Lucas Nascimento
Implementar lógica de atualização do status de pedido e exibição para o Cliente	antonio neri
Painéis Interativos (Entregador e Restaurante)	Johnatan Rodrigues
Implementar tela de visualização de pedidos pendentes para o Restaurante (CLI).	Johnatan Rodrigues
Implementar lógica para o Restaurante confirmar ou rejeitar um pedido.	Johnatan Rodrigues
Implementar tela de visualização de pedidos disponíveis para o Entregador (CLI).	Johnatan Rodrigues
Implementar lógica para o Entregador aceitar ou recusar uma entrega.	Johnatan Rodrigues

Figura 7

Durante a Sprint 6, a equipe concentrou seus esforços em aprimorar a experiência conversacional do módulo de delivery por meio da interface da linha de comando (CLI). O foco foi na interatividade do sistema, abrangendo desde fluxos de aceite e recusa de pedidos pelos estabelecimentos e entregadores até um sistema completo de avaliações e acompanhamento de rota, enriquecendo o fluxo operacional para todas as partes envolvidas. Todas as atividades planejadas foram concluídas com sucesso.

As principais entregas desta sprint foram:

- **Sistema de Avaliações:** Criação das entidades, dados e repositórios necessários para hospedar avaliações e análises (reviews). As interfaces via CLI foram desenvolvidas para suportar a avaliação mútua entre cliente, restaurante e entregador após a conclusão de cada entrega.
- **Acompanhamento e Rota de Entrega:** Implementação da lógica de visualização e simulação da rota com mapa para o entregador diretamente na CLI. Em paralelo, foi incluída a atualização assíncrona/dinâmica do status do pedido, exibindo a evolução da entrega ao Cliente.
- **Painéis Interativos:** Estruturação avançada das telas de pedidos para Entregadores e Restaurantes. Foi incluída a listagem de solicitações pendentes e a

lógica que de fato permite aceitar ou rejeitar uma solicitação via interação no console
- **Testes:** Extensão considerável na segurança do código, englobando novos testes unitários para a entidade recém-criada de Avaliações, baterias de testes de integração atuando na camada de aceite/recusa de pedidos e refinamento constante dos testes projetados em sprints anteriores.
- **Documentação**: Atualização técnica fundamental detalhando todo o fluxo introduzido recentemente: novos diagramas de classes, diagramas de casos de uso e de atividades, além da condução dos relatórios da recém-lançada release.

A sprint foi finalizada com todas as atividades concluídas de forma robusta, cimentando lógicas avançadas de pedido e transformando o backend do sistema em uma verdadeira simulação dinâmica e responsiva.

- **Papéis:**
  - **Antônio Neri:** Desenvolvedor
  - **Johnatan Rodrigues:** Gerente, Desenvolvedor
  - **Lucas Nascimento**: Desenvolvedor, Testador

Como foi implementado (Sistema de Avaliações e Acompanhamento): As avaliações foram estabelecidas modelando a entidade Avaliacao, e o ciclo de feedback foi incorporado utilizando fluxos de Scanner injetados nas respectivas Views (ex: `ClienteView`, `RestauranteView`). Por outro lado, para o acompanhamento e roteirização, a lógica de atualização dinâmica dos status percorre o ciclo de vida do pedido gerenciado pelo `PedidoService`, disparando visualizações (podendo ser em formato textual/mapa via string) que dão o indicativo de direção e status para o passageiro/entregador.

Como foi implementado (Painéis Interativos e Rota de Entrega): O controle de demandas foi instanciado utilizando os menus para Entregador e Restaurante (CLI). Ao o restaurante ou o entregador executar a ação afirmativa/negativa em relação a uma entrega, os controladores enviam a instrução para o `PedidoService`, onde o atributo de status do rastreamento é modificado dentro do repositório respectivo, disparando atualizações refletidas do lado do cliente e viabilizando o teste de integração completo de uma demanda ao aceitá-la ou recusá-la.